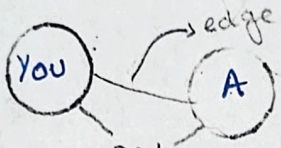
Q1 Graph representation

1. Adjacency matrix
2. Adjacency list

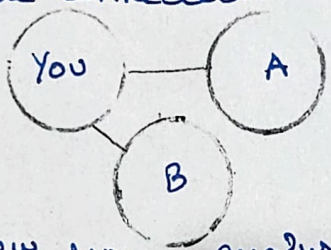
Before we talk about adjacency list and matrix lets talk about what a graph is. A graph is a network of nodes, these nodes can represent anything from people, to places, products, objects, servers etc. Lets think of an example if a graph is a social network and nodes people, then people who are friends will be connected to each other using lines (edges).
lets visualise this. If you were a node →



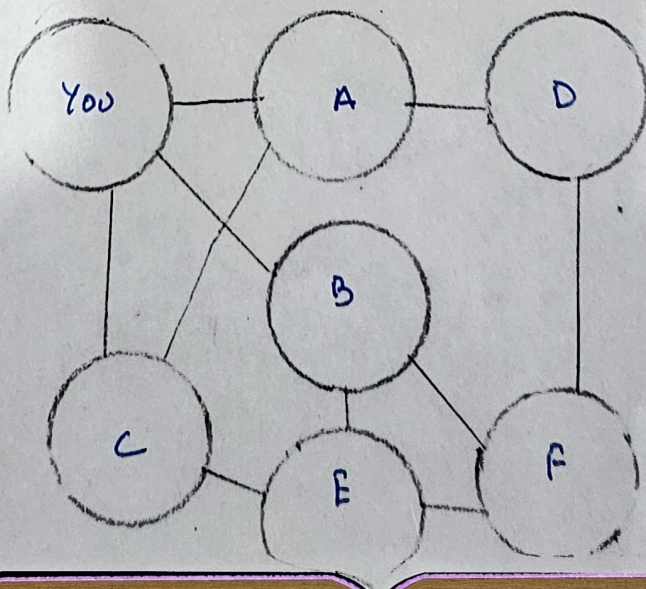
And now lets say you made a new friend lets call him A now you both will be connected by an edge —



Now you've made another friend, B now both of you will again be connected but since A and B aren't friends they won't be connected.



Now, we know how graphs work but what are they used for? lets say your friend group grew large.
You've lot more friends and so does, A and B



Now, we have a problem, let's say you want to be friends with 'f' ~~how~~ and to be friends with F the only way is to be introduced by someone You have three options:-

1. Ask A to introduce to D → D introduces You to F → 2 edges
2. Ask B to introduce you to F → 1 edge
3. Ask C to introduces to E → E introduces to F → 2 edges.

we count the number of edges (weight) and find the fastest way is to go through B. This is exactly how graphs are used.

1. Navigation → Google/Apple maps
2. Recommendation → Amazon/Flipkart, Netflix Jio hotstar.
3. Social Network

Now, we move to Adjacency matrix, Adjacency matrix is like a spreadsheet ~~or~~ where both rows and columns are nodes if a node is connected to other their cell reads 1 else it reads 0 for our earlier graph here is how Adjacency matrix would look like:-

	You	A	B	C	D	E	F
You	0	1	1	1	0	0	0
A	1	0	0	1	1	0	0
B	1	0	0	0	0	1	1
C	1	1	0	0	0	1	0
D	0	1	0	0	0	0	1
E	0	0	1	0	0	0	1
F	0	0	1	0	1	1	0

2. Adjacency list → An adjacency list is like a book, each node is a page in this book and each page has a list of friends they have for example F's page would have a list like this → B → D → E → NULL, The whole graph we made earlier would look like this

You: A → B → C
A: You → C → D
B: You → B → F
C: You → A → E
D: A → F
E: B → F
F: B → D → E

Now that we have a idea of how these look now, we talk about:

b) Comparision of memory requirement for sparse vs Dense graphs.

i) Sparse graph: if we had a huge network of 1000 people but only 8 friendship in total then we would have a matrix of 1000 × 1000 grid so → 1000000 cells and since we have only 8 friendships only 16 cells out of 1000000 will be filled giving us 999984 empty cells. Now what about Adjacency list. The list would have 1000 pages and 8 edges = 16 pieces of data that is 1016 pieces of data compared to 1000000.

ii) Dense graph: lets say we have a dense graph where almost/everyone is a friend of other, if we have n nodes the number of edges is around n^2 now when this happens a matrix is better because:

1. Matrix is fixed a 1000 × 1000 matrix will always have 1000000 data points storing 0 or 1 takes 1 bit.

2. List becomes bloated → we use nodes pointer in a list now imagine if these nodes can have many friends each node would be duplicating the whole 12-bit node causing a big bloat in the memory.

c) Time complexity

i) Checking if edge exists between two vertices

- Matrix view → if we had to check if Node 999 and node 52 are friends fastest way would just an look up matrix[42] this is instant so O(1) time

- List view → we can first move to list[42], but if 42 has many friends eg. 500 friends then we would read through every name which will take O(n) where n is the number of vertices.

ii) Finding all neighbours of a vertex

- Matrix → we will have to read entire row size V to see who is 1 and who is 0 so O(n).

- List → we just need to open the list and read it so O(n) where n is no. of friends. So list is faster.

iii) Adding new edge

- Matrix → we go to row u and column v and turn 0 to 1 and then row v column u and turn 0 to 1 so O(1).

- List → go to page u write v and go to page v write u so O(1). Both take same time.

iv) Adding new vertex

- Matrix → we add a new row and a new column so $O(V^2)$

→ List → we add a new list so O(1).

Code implementation →

```
#<stdio.h>
#<stdlib.h>
struct node {
    int data;
    struct node *nxt;
}; typedef struct node node;

struct graph {
    int numVertices;
    node **adjList;
}; typedef struct graph graph;

graph* createGraph(int vertices) {
    graph* myGraph = (graph*) malloc(sizeof(graph));
    myGraph->numVertices = vertices;
    myGraph->adjList = (node**) malloc(vertices * sizeof(node*));
    for(int i=0; i<vertices; i++) {
      myGraph->adjList[i] = NULL;
    } return myGraph;
}
Void addEdge(graph *mygraph, int src, int dest)
{ node *newnode = (node *) malloc(sizeof(node));
  newnode->data = dest;
  newnode->nxt = myGraph->adjList[src];
  myGraph->adjList[src] = newnode;

  node *newnode2 = (node *) malloc(sizeof(node));
  newnode2->data = src;
  newnode2->nxt = myGraph->adjList[dest];
  myGraph->adjList[dest] = newnode2;
}
void printGraph(graph *myGraph){
  for(int i=0; i<myGraph->numVertices; i++)
{ node *temp = myGraph->adjList[i];
  printf("Vertice %d:", i);
  while(temp != NULL)
{ printf("-> %d", temp->data);
  temp= temp->nxt;
} printf("-> NULL\n");
}
}
```

```
int main()
{ graph *mygraph = createGraph(5);
  addEdge(mygraph, 0, 1);
  addEdge(mygraph, 1, 2);
  addEdge(mygraph, 1, 4);
  addEdge(mygraph, 2, 3);
  addEdge(mygraph, 3, 4);
  printGraph(mygraph);
  return 0;
}
```

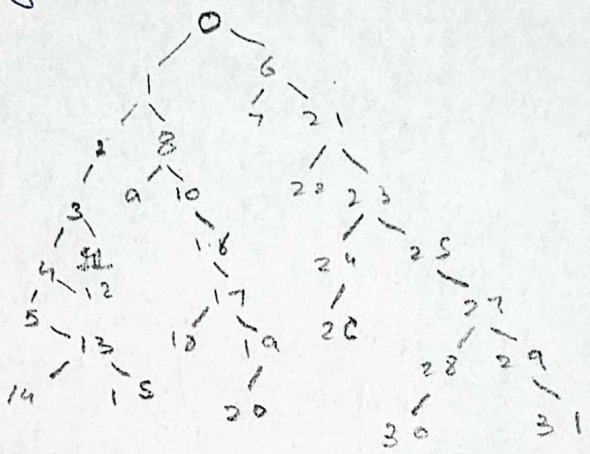
MatrixView

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
  int numVertices;
  int** adjMatrix;
} MatrixGraph;
MatrixGraph* createMatrixGraph(int vertices)
{ MatrixGraph* graph = (MatrixGraph*) malloc(sizeof(MatrixGraph));
  graph->numVertices = vertices;
  graph->adjMatrix = (int**) malloc(vertices * sizeof(int*));
for (int i=0; i<vertices; i++) {
  graph->adjMatrix[i] = (int*) malloc(vertices * sizeof(int));
  for (int j=0; j<vertices; j++) {
    graph->adjMatrix[i][j] = 0;
  }
} return graph;
}
void addMatrixEdge(MatrixGraph* graph, int src, int dest) {
    graph->adjMatrix[src][dest] = 1;
    graph->adjMatrix[dest][src] = 1;
}
  void printMatrixGraph(MatrixGraph* graph) {
    for(int i=0; i<graph->numVertices; i++)
    { for(int j=0; j<graph->numVertices; j++)
        printf("%d ", graph->adjMatrix[i][j]);
      printf("\n");
    }
  }
```

(Q2) Depth-first Search (DFS)

a) Depth-first search is the strategy of traversal through a graph/tree. It has works in a way that imagine we have a huge physical maze and at every fork we take the left route until we reach the end, once we reach the end, we backtrack to the latest fork and choose the unvisited route and then follow the same routine, reaching every possible room.

Example walkthrough:



In the above example tree we will start at our chosen vertice (0 in this case) and we will keep going left while printing the visited node.

→ 0 → 1 → 2 → 3 → 4 → 5

Now there is no more left from 5 so we take a right

5 → 13, now 13 has a left again so we go left again

5 → 13 → 14, 14 is a dead end so, we now go to 15

5 → 13 → 14 → 15, 15 is a dead end again so we backtrack:

we see 4 has a child so we visit this child

13 → 14 → 15 → 12, We backtrack again 3 has another child.

14 → 15 → 12 → 11, backtrack again 1 has a child so,

15 → 12 → 11 → 8 → 9 → 10 → 16 → 17 → 18 → 19 → 20 → 6 → 7 → 21 → 22 → 23 → 24 → 26 → 25 → 27 → 28 → 30 → 29 → 31, the whole walkthrough of us visiting every node.

Step-by-step working

1. Start at a chosen vertex
2. Mark the vertex as visited
3. Look at all neighbours of this vertex
4. If a neighbour has not been visited, pause the current node and immediately run DFS on that neighbour.
5. Once all neighbours have been visited, backtrack to the node that called this one.

b) Recursive & iterative Implementations

- Recursive DFS → This uses computer's built in call stack.
  - When DFS() is called inside DFS(), the computer pauses the first call saves its state and puts it on a stack in memory.
  - Incredibly clean, short, it allows for a shorter quicker code.
  - Its con includes a problem with memory, if the graph is millions of nodes deep, system eventually runs out of memory causing stack overflow crash.
- Iterative DFS → In order to implement this we create our own stack structure using arrays or linked list.
  - For this we push nodes to our own stack and use while(stack is not empty) loop to pop them off and visit them.
  - It allows us to check massive graphs as it uses heap memory.
  - The code is a lot longer compared to Recursive DFS due to a need of managing the stack manually.

c) Time and space complexity

- Time Complexity → We visit each vertex exactly once so V we also check every single edge to see if a vertex has been visited. So the time complexity becomes O(V+E)

- Space Complexity → we need an array of size V to keep track of the visited nodes. In worst-case, the stack will hold all V vertices at the same time so O(V).

d) Applications of DFS

1. Maze Solving/Path finding: DFS finds a path to finish in a maze although not the shortest way but a path nonetheless.
2. Finding strongly/weakly connected components of a graph: DFS can help find a component that has many paths from each vertex in the graph (strongly connected) or the ones which have the least paths connected to them (weakly connected).

3. Topological Sorting: Used to schedule tasks with dependencies (eg. How we need to take AEM-I before AEM-II).
4. Webcrawlers: DFS can be used in the implementation of web crawlers to explore links on a website

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct node {
    int data;
    struct node *next;
} node;
typedef struct graph {
    int numVertices;
    node **adjList;
} graph;
// Create graph and Add edge functions from last question

bool DFS_Recursive ( graph* g, int vertex, bool visited[], int parent)
{
  visited[vertex] = true;
  printf ("%d ", vertex);

  bool hasCycle = false;
  node* temp = g->adjList[vertex];
  while (temp != NULL) {
    int connectedVertex = temp->data;
    if(!visited[connectedVertex]) {
      if (DFS_Recursive (g, connectedVertex, visited, vertex)) {
        hasCycle = true;
      }
    } else if (connectedVertex != parent) {
      hasCycle = true;
    } temp = temp->next;
    return hasCycle;
  }

void DFS_Iterative (graph* g, int StartVertex, bool visited[]) {
  int Stack [g->numVertices];
  int top = -1;

  Stack [++top] = StartVertex;
```

```
while (top >= 0) {
    int current = stack[top--];
    if (!visited[current]) {
        printf("%d ", current);
        visited[current] = true;
    }
    node* temp = g->adjList[current];
    while (temp != NULL) {
        int adj = temp->data;
        if (!visited[adj]) {
            stack[++top] = adj;
        }
        temp = temp->next;
    }
}
}

void findComponentAndDFS(Graph* g) {
    bool visited[g->numVertices];
    for (int i=0; i<g->numVertices; i++) {
        visited[i] = false;
    }
    int components = 0;
    bool containsCycle = false;
    for (int i=0; i<g->numVertices; i++)
    {   if(!visited[i]){
        components++;
        printf("Component %d: ", components);
        if(DFS_Recursive(g, i, visited, -1))
        {containsCycle = true;
        }
        printf("\n");
    }
}
printf("\nTotal Connected Components = %d\n", components);
}
}
```

Manual Trace

Vertices 0, 1, 2, 3, 4, 5

Edges 0-1, 1-2, 2-0, 3-4, 4-5

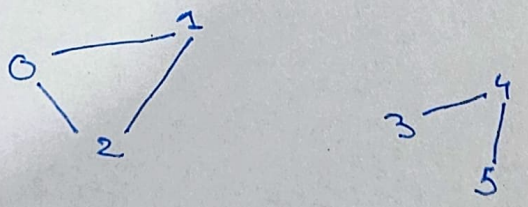
1. main loop Starts at i=0. Not visited, prints "Component 1"
2. DFS (0, Parent = -1) :→ 0 visited Print 0.
   Neighbour 1 unvisited
3. DFS (1, Parent = 0) → 1 visited Print 1
   Neighbour 0 is visited, but it is equal to Parent so Ignores,
   Neighbour 2 unvisited
4. DFS (2, Parent = 1) : 2 visited Print 2
   Neighbour 1 visited equals Patient Ignore
   Neighbour 0 is visited And it does not equal parent(1)
   Back - edge found hasCycle = True.
   Backtrack to 1 → Backtrack to 0 → component 1 finished.
5. Main Loop : Checks 1, 2 Already visited Reaches 3 Not visited Print Component 2
6. DFS (3, Parent = -1) Mark 3 visited Print 3
   Neighbour 4 is unvisited
7. DFS (4, Parent = 3): Mark 4 visited → Print 4
   Neighbour 5 unvisited
8. DFS (5, Parent = 4): Mark 5 visited → Print 5
   Dead End Backtrack <-> 4 -> 3 → Component 2 finished
9. Mainloop → Check 4, 5 (Already visited) loop ends

Output

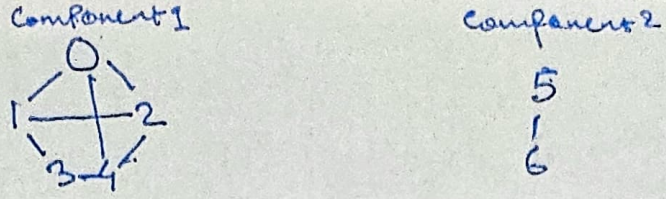
Component 1 : 0 1 2

Component 2 : 3 4 5

Total connected component : 2



Graph



a) BFS relies on an Queue (First In/First Out) System to keep track of which nodes to visit next.

Step-by-Step working

1. Initialize: We first create an empty Queue, a visited array to keep track of visited nodes and distance array to check distance.
2. Start: Enqueue the first vertex mark it visited and set distance to 0.
3. Loop: while the Queue is not empty
   - Dequeue the front vertex → name it current
   - Look at all neighbours of current
   - For every neighbour that has not been visited
     - Mark it as visited
     - Set its distance: distance[neighbour] = distance[current] + 1.
     - Enqueue the neighbour.
4. Repeat until the Queue is empty

b) BFS VS DFS

	BFS	DFS
1.	Explores level by level	Explores path-by-path
2.	Uses a Queue	uses a Stack
3.	Guarantees the shortest path	Does not Guarantee shortest path
4.	O(V) but can be memory heavy if the graph is wide	O(V) but can be cause stack overflow if the graph is too deep.
5.	Used for GPS, Social network connections	Maze Solving, cycle detection, topological Sorting.

c) Finding Connected Components with BFS

1. Create a visited array for all vertices
2. Loop through every vertex i from 0 to V-1
3. If visited[i] is false, a new component is found.
4. Run standard BFS from i. It will mark all nodes connected to i as visited.
5. When BFS finishes. The number of times BFS was triggered is the number of components.

8) #include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct node {
    int data;
    struct node *next;
} node;

typedef struct graph {
    int numVertices;
    node **adjList;
} graph;

```
Man=100;
int queue[Man];
int front=-1, rear=-1;

void enqueue(int vertex){
    if(front==-1) front=0;
    queue[++rear]=vertex;
}
int dequeue(){
    return queue[front++];
}
bool isQueueEmpty(){
    return front==-1 || front>rear;
}
void resetQueue(){
    front=-1;
    rear=-1;
}
void BFS(graph* g, int startVertex, bool visited[], int distance[],
    int parent[])
{   visited[startVertex]=true;
    distance[startVertex]=0;
    parent[startVertex]=-1;

    enqueue(startVertex);
    while(!isQueueEmpty()){
        int current=dequeue();
        printf("%d ", current);

    node* temp=g->adjList[current];
    while(temp!=NULL){
    int neighbour=temp->data;
    if(!visited[neighbour]=true;
    distance[neighbour]=distance[current]+1;
    parent[neighbour]=current;
    enqueue(neighbour);
}temp=temp->next;
```

```
}
}
}
void printPath(int parent[], int destination) {
  if(parent[destination] == -1){
    printf("%d", destination);
    return;
  }
  printPath(parent, parent[destination]);
  printf("-> %d", destination);
}
void analyzeGraphBFS(graph* g, int startVertex, int targetVertex) {
  bool visited[MAX] = {false};
  int distance[MAX] = {0};
  int parent[MAX];
  int components=0;
  for(int i=0; i<g->numVertices ; i++) {
    if(!visited[i]) {
      components++;
      printf("Component %d: ", components);
      resetQueue();
      BFS(g, i, visited, distance, parent);
      printf("\n");
    }
  }
  printf("Total connected components %d\n", components);
    for(int i=0; i<g->numVertices; i++) {
      if(distance[i]>0 || i == startVertex) {
        printf("Distance to %d : %d edges\n", i, distance[i]);
      }
    }
    printf("Actual path from %d to %d: ", startVertex, targetVertex);
  printPath(parent, targetVertex);
  }
```

f) Manual Tracing

- Vertices (7): 0, 1, 2, 3, 4, 5, 6
- Edges (8): 0-1, 0-2, 1-2, 1-3, 0-4, 2-4, 3-4, 5-6

1 → Enqueue 0.

2. Dequeue 0 Print 0.
   - Neighbours of 0: 1, 2, 4 All unvisited
   - Mark visited. Distance: d = 1, d = 1, d = 1.
   - Parent P=0, P=0, P=0.
   - Enqueue. Queue: ".

3. Dequeue 1: Print 1
   - Neighbours: 0, 2, 3 (0 & 2 are already visited).
   - unvisited neighbour 3
   - mark visited d = d+1 = 2. P=1
   - Enqueue. Queue: ".

4. Dequeue 2: Print 2
   - Neighbours 0, 1, 4 All visited
   - Queue

5. Dequeue 4 Print 4
   - Neighbours: 0, 2, 3 All visited. DO nothing.
   - Queue: ".

6. Dequeue 3: Print 3
   - Queue is empty, Component finished

7. Main loop: Jumps to node 5 (unvisited). Enqueue 5.

8. Dequeue 5 Print 5
   - Neighbour: 6
   - Mark visited d=1, P = 5 enqueue 6.

9. Dequeue 6: Print 6 Done.

Question 4

Minimum Spanning Tree

→ A spanning tree is a subgraph that includes every single vertex of the original graph, it is fully connected, and contains zero cycles. If the graph has V vertices, a spanning tree always has exactly V-1 edges.

→ Minimum Spanning Tree (MST): Out of all possible spanning tree you could draw, the MST is the one where the sum of the edges weights is the absolute smallest.

A MST cannot ~~contain~~ exist if the graph is disconnected there is an isolated island of node.

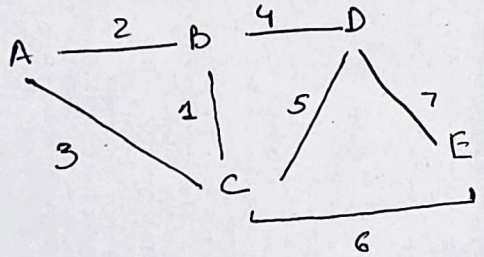
b) Prim's Algorithm

Prim's Algorithm is a greedy algorithm. It starts at an random node. It looks at every edge connected to the current tree, and absorbs the cheapest one that connects to an unvisited node. It repeats this until all nodes are absorbed.

Time Complexity: using Adjacency Matrix: $O(V^2)$

using Adjacency list + binary Min-heap: $O(E \log V)$

Sample Graph



Taking A as starting node

visited: {A}

unvisited: {B, C, D, E}

A-B cost 2

A-C cost 3

A-B - cheapest - we take it

Add to MST: edge (A-B)

visited: {A, B}

unvisited: {C, D, E} → Now it takes B

(A-C) cost: 3

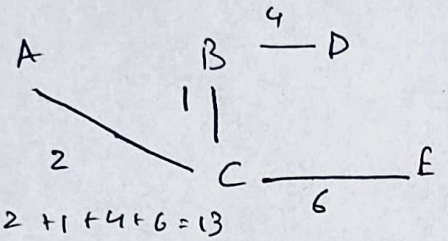
(B-C) cost 1

(B-D) cost: 4

(B-C) → cheapest we take it

Step 2

visited {A, B, C}
unvisited {D, E}
Now we are on C
B-D cost: 4 → cheapest we take this
C-D cost 5
C-E cost 6
visited {A, B, C, D}
unvisited {E}
Now we are on D
C-E cost 6 — cheapest we take this
D-E cost 7
visited {A, B, C, D, E}
All visited.
Final MST:



Total cost : 2 + 1 + 4 + 6 = 13

c) Kruskal's Algorithm

Kruskal's Algo takes a look at the entire map at once, sorts the edges from cheapest to most expensive. It goes down the list picking the cheapest edge available. If adding an edge creates a cycle it discards it. This continues till it has V-1 edge

Time complexity : O(E LogE)

d)

Prim's	Kruskal's
Starts at one node grows as a single continuous tree	starts every node builds a forest that eventually merges into one tree.
Priority Queue Min-heap	Edge list, union find

) Edges
(A-B)4, (A-C)3, (B-C)1, (B-D)2, (C-E)5, (D-E)6, (D-F)7, (E-F)8

Prim's (Starting at A)

1. Start A : (A-C)3, (A-B)4 → (A-C)3 visited A, C
2. From {A, C} : (A-B)4, (C-B)1, (C-E)5 → (B-C)1 visited A, B, C
3. From {A, B, C} : (A-B)4, (B-D)2, (C-E)5 → (B-D)2 visited A, B, C, D
4. From {A, B, C, D} : (C-E)5, (D-E)6, (D-F)7 → (C-E)(5) visited ABCDE
5. From {A, B, C, D, E} : (D-F)(7) → ~~D-F~~ visited ALL

Total weight = 3+1+2+5+7 = 18

Kruskal

1. Sorted : (B-C)1, (B-D)2, (A-C)3, (A-B)4, (C-E)5, (D-E)6, (D-F)(7), (E-F)8
2. Pick B-C 1 → No cycle
3. B-D (2) No cycle
4. A-C (3)
5. A-B(4) Rejected cycle at A-C-B
6. (C-E) 5
7. D-E 6 Rejected cycle at (C-E-D-B)
8. D-F(7) No cycle Tree has 5 Edges stop.

Total weight = 1+2+3+5+7 = 18

Code

```
# include <stdio.h>
# include <stdlib.h>
# include <stdbool.h>
INF = 9999;
// Prims
int extractMin (int key[], bool mstSet[]) {
  int min = INF, min_index;
  for (int v=0; v<V; v++) {
    if (mstSet[v] == false && key[v] < min) {
      min = key[v];
      min_index = v;
    }
  } return min_index;
}
void primMST (int graph[V][V]) {
  int parent[V];
  int key[V];
  bool mstSet[V];
```

```
for(int i=0; i<V; i++){
    key[i] = INF;
    mstSet[i] = false;
}
    key = 0;
    parent = -1;
for (int count=0; count < V-1; count++) {
    int u = extractMin(key, mstSet);
    mstSet[u] = true;
for (int v=0; v<V; v++) {
  if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]){
    parent[v] = u;
    key[v] = graph[u][v]
}
}
}
int totalWeight = 0;
for (int i=1; i<V; i++){
  printf("%c - %c  %d\n", parent[i] + 'A', i + 'A', graph[i][parent[i]]);
  totalWeight += graph[i][parent[i]];
printf("Total weight : %d\n", totalWeight);
}

// Kruskal's Algorithm
typedef struct ~~Edge~~ { Subset {
        int parent;
        int rank;
    } subset;
int find(subset subset[], int i){
    if (subset[i].parent != i){
      subsets[i].parent = find(subsets, subsets[i].parent);
  }
  return subsets[i].parent;
}
void Union(subset subsets[], int x, int y){
        int rootX = find(subsets, x);
        int rootY = find(subsets, y);
    if (subsets[rootX] < subsets[rootY].rank){
      subsets[rootX].parent = rootY;
```

a) Single-Source Shortest Path - find the path with minimum total weight from one specific starting vertex to every other vertex in the graph.

b) Dijkstra's Algorithm is a greedy Algorithm it explores outward like a water droplet expanding on a paper - always prioritizing the cheapest known destination first.

```
function Dijkstra (Graph, source):
    Create distance array
    Create parent array
    distance [source] = 0
Create a priority Queue (Min-Heap) and insert (0, source)
while Priority Queue is not empty:
    current_node = pop node with minimum distance
    if current_node is already visited, continue
        mark current_node as visited
    for each neighbour of current_node:
        new_distance = distance [current_node] + weight (current_node,
        neighbour)
        if new_distance < distance [neighbour]:
            distance [neighbour] = new_distance
            parent [neighbour] = current_node
            push (new_distance, neighbour)
```

c) Step-by-Step Manual Execution

Edges (0,1) 4, (0-2) 1, (2-1) weight 2, (1-3) 1, (2-3) 5, (3-4) 3, (2-4) 8, (0-4) 10

Step	Current node	Distance Array	Priority Queue	Action / Relaxation
0	-	0, ∞	[(0,0)]	initialization
1	0	[0, 4, 1, ∞, 10]	[(1,2), (4,1), (10,4)]	Relax 0→2 0→1 0→4
2	2	[0, 3, 1, 6, 9]	[(3,1), (4,1), (6,3), (9,4)...]	Relax 2→1 2→3 2→4
3	1	[0, 3, 1, 6, 9]	[(4,3), (4,1), (6,3), (9,4)...]	Relax 1→3
4	3	[0, 3, 1, 6, 9]	[(7,4) (9,4), (10,4)]	Relax 3→4
5	4	[0, 3, 1, 6, 9]	[empty]	No outbound edges Algorithm finished.

d)

Adjacency Matrix + array : $O(V^2)$ -> Requires scanning the entire array of V vertices V time

Adjacency List + Binary Min-Heap : $O((V+E)\log V)$ -> Extracting the min node takes $O(\log V)$ we do this V times. Updating neighbours takes $O(\log V)$ we do it E times.

Adjacency list + Fibonacci Heap: $O(E+V\log V)$ -> Fibonacci $O(1)$ constant time Extract-min heap -> $O(\log V)$

e) Limitations: Dijkstra's algorithm assumes that adding an edge to a path always makes the path longer or keeps it the same. If you have a negative edge, Dijkstra's greedy logic breaks.

f)
```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
Max_vertices = 100;
typedef struct AdjNode {
    int dest;
    int weight;
    struct AdjNode* next;
} AdjNode;

typedef struct MinHeapNode {
    int vertex;
    int dist;
} MinHeapNode;

void printPath(int parent[], int j) {
    if (parent[j] == -1) return;
    printPath(parent, parent[j]);
    printf("-> %d", j);
}

Void dijkstra(AdjNode* graph[], int V, int source) {
    int dist[Max_Vertices];
    int parent[Max_vertices];
    int visited[Max_vertices] = {0};
    for (int i = 0; i < V; i++) {
        dist[i] = INT_Max;
        parent[i] = -1;
    }
    dist[source] = 0;
    for (int count = 0; count < V-1; count++) {
        int min_dist = INT_Max, u = -1;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] <= min_dist)
```

```
        min_dist = dist[v];
            u = v;
    }
}
    visited[u] = 1;
AdjNode* crawl = graph[u];
  while(crawl != NULL){
    int v = crawl->dest;
    if(!visited[v] && dist[u] != INT_MAX && dist[u] + crawl->weight < dist[v])
    { dist[v] = dist[u] + crawl->weight;
      parent[v] = u;
    }
    crawl = crawl->next;
  }
}  for(int i=0; i<V; i++){
printf("\n%d -> %d \t %d \t\t %d", source, i, dist[i], source);
    printPath(parent, i);
  }
}
```

Algorithm	Purpose	Data Structure	Time Complexity	Space Complexity	Graph	Application
DFS	Deep exploration, cycle detection	Stack	O(V+E)	O(V)	unweighted	Maze solving
BFS	Shortest path by number of edges	Queue	O(V+E)	O(V)	unweighted	Web crawler
Prim's	MST	Priority Queue	O((V+E) log V)	O(V)	weighted	Network cabling
Kruskal	MST	Disjoint Set	O(E log E)	O(V)	weighted	MSTs in sparse graphs
Dijkstra's	Shortest Path	Priority Queue	O(V+E) log V)	O(V)	weighted	Google maps

) Problem-Solving

(A-B)2, (A-C)4, (B-C)1, (B-D)(5), C-D3, C-E6, D-E2

1. BFS
   - Path to C: A→C 1 edge
   - Path D: A→B→D 2 edges
   - Path to E: A→C→E 2 edges

2. Dijkstra
   Path to C: A→B→C weight 2+1=3
   Path to D: A→B→C→D (weight 2+1+3=6)
   Path to E: A→B→C→D→E (weight 2+1+3+2=8)

3. Prim's MST
   - Pick A-B(2)
     B-C(1)
     C-D(3)
     D-E(2)
   MST weight = 8

4. Kruskal's MST
   - Sort (B-C)1, (A-B)2, D-E(2), C-D(3), A-C(4), B-D(5), C-E(6)
   - Pick (B-C), (A-B), (D-E), (C-D)

   Total weight 8.

Although Dijkstra's & MST are same they are fundamentally different
MST: wants to connect every city using lowest amount of weight possible.
SSSP: wants to go from A to E as fast as possible